



**THE UNIVERSITY OF AZAD JAMMU AND
KASHMIR, MUZAFFARABAD**
Department of Software Engineering



STUDENT NAME:

SHAHZAD AHMED AWAN

STUDENT ROLL NO:

2024-SE-15

LAB:

Open ended Lab

COURSE TITLE:

Data Structure & Algorithm

COURSE CODE:

CS-2101

INSTRUCTOR:

Engr. Asma Javed

SUBMISSION DATE:

16th January, 2026

Table of Contents

OPEN ENDED LAB (OEL)	3
Lab Title	3
Lab Type	3
Objective	3
Tools & Technologies	3
Theoretical Background.....	3
Types of Graphs	3
Graph Representations.....	4
Graph Traversal Algorithms.....	4
Implementation Details.....	4
Program 1: Graph Representation using Adjacency Matrix:	4
Program 2: Graph Representation using Adjacency List	6
Program 3: Breadth First Search (BFS) Traversal.....	8
Program 4: Depth First Search (DFS) Traversal.....	10
Results & Observations	12
Conclusion	12
Reflection	12

Table of Figure:

Figure 1: Graph representation using adjacency matrix Code -1.....	4
Figure 2: Graph representation using adjacency matrix Code -2.....	5
Figure 3: Graph representation using adjacency matrix Output.....	5
Figure 4: Graph Representation using Adjacency List Code -1.....	6
Figure 5: Graph Representation using Adjacency List Code- 2.....	7
Figure 6: Graph Representation using Adjacency List Output.....	7
Figure 7: Breadth First Search (BFS) Traversal Code -01.....	8
Figure 8: Figure 8: Breadth First Search (BFS) Traversal using queue Output.....	9
Figure 9: Figure 8: Breadth First Search (BFS) Traversal using queue Code -02	9
Figure 10: DFS Traversal using Recursion Code -01.....	10
Figure 12: DFS Traversal using Recursion Output	11
Figure 11: DFS Traversal using Recursion Code -02.....	11

OPEN ENDED LAB (OEL)

Lab Title

Graph Data Structure Implementation in C++

Lab Type

Open Ended Lab (Self-Learning & Implementation)

Objective

The objective of this Open-Ended Lab (OEL) is to independently study and implement the Graph data structure in C++. This lab aims to strengthen understanding of graph concepts, graph representations, and traversal algorithms such as Breadth First Search (BFS) and Depth First Search (DFS) through practical implementation.

Tools & Technologies

- **Programming Language:** C++
- **Compiler/IDE:** Dev C++

Theoretical Background

A graph is a non-linear data structure consisting of vertices (nodes) and edges (connections). Graphs are widely used to model real-world problems such as computer networks, road maps, and social networks.

Types of Graphs

- **Directed Graph:** Edges have direction
- **Undirected Graph:** Edges have no direction
- **Weighted Graph:** Edges contain weights or costs

Graph Representations

- **Adjacency Matrix:**
A 2D array representation where 1 shows an edge and 0 shows no edge.
- **Adjacency List:**
Each vertex maintains a list of its adjacent (connected) vertices.

Graph Traversal Algorithms

- **Breadth First Search (BFS):** Traverses the graph level by level using a queue.
- **Depth First Search (DFS):** Traverses the graph deeply using recursion or stack.

Implementation Details

Program 1: Graph Representation using Adjacency Matrix:

This program represents a graph using a 2D adjacency matrix. The user provides the number of vertices, edges, and their connections.

Code:

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int vertices, edges;
7      cout << "===== "<<endl;
8      cout << "          Graph Representation using adjancy Matix"<<endl;
9      cout << "===== "<<endl;
10     // Input number of vertices
11     cout << "Enter number of vertices (1 to 20): ";
12     cin >> vertices;
13     if (vertices <= 0 || vertices > 20)
14     {
15         cout << "Invalid number of vertices!" << endl;
16         return 0;
17     }
18     // Initialize adjacency matrix
19     int graph[21][21] = {0}; // Using 1-based indexing
20     // Input number of edges
21     cout << "Enter number of edges: ";
22     cin >> edges;
23     // Maximum edges in undirected graph with self-loops = n*(n+1)/2
24     if (edges < 0 || edges > vertices * (vertices + 1) / 2)
25     {
26         cout << "Invalid number of edges!" << endl;
27         return 0;
28     }
```

Figure 1: Graph representation using adjacency matrix Code -1

```

29 // Input edges
30 for (int i = 0; i < edges; i++)
31 {
32     int u, v;
33     cout << "Enter edge " << i + 1 << " (vertex1 vertex2): ";
34     cin >> u >> v;
35
36     // Validate vertex numbers
37     if (u < 1 || v < 1 || u > vertices || v > vertices)
38     {
39         cout << "Invalid vertices! Enter values between 1 and " << vertices << endl;
40         i--; // retry this edge
41         continue;
42     }
43
44     // Undirected graph
45     graph[u][v] = 1;
46     graph[v][u] = 1;
47 }
48
49 // Display adjacency matrix
50 cout << "\nAdjacency Matrix Representation:\n\n ";
51 for (int i = 1; i <= vertices; i++)
52     cout << "V" << i << " ";
53 cout << endl;
54
55 for (int i = 1; i <= vertices; i++)
56 {
57     cout << "V" << i << " ";
58     for (int j = 1; j <= vertices; j++)
59     {
60         cout << graph[i][j] << " ";
61     }
62     cout << endl;
63 }
64
65 return 0;
66 }

```

Figure 2: Graph representation using adjacency matrix Code -2

Output:

```

C:\Users\hp\Desktop\2024-SE  X  +  v
=====
Graph Representation using adjancy Matix
=====
Enter number of vertices (1 to 20): 5
Enter number of edges: 4
Enter edge 1 (vertex1 vertex2): 2 1
Enter edge 2 (vertex1 vertex2): 2 2
Enter edge 3 (vertex1 vertex2): 3 1
Enter edge 4 (vertex1 vertex2): 5 1

Adjacency Matrix Representation:

  V1 V2 V3 V4 V5
V1 0  1  1  0  1
V2 1  1  0  0  0
V3 1  0  0  0  0
V4 0  0  0  0  0
V5 1  0  0  0  0

-----
Process exited after 19.82 seconds with return value 0
Press any key to continue . . .

```

Figure 3: Graph representation using adjacency matrix Output

Program 2: Graph Representation using Adjacency List

This program represents a graph using an adjacency list, where each vertex stores a list of connected vertices.

Code:

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main()
6  {
7      int vertices, edges;
8      cout << "===== "<< endl;
9      cout << "          Graph Representation using Adjancey list  "<< endl;
10     cout << "===== "<< endl;
11     // Input number of vertices
12     cout << "Enter number of vertices (1 to 20): ";
13     cin >> vertices;
14
15     // Validate vertices
16     if (vertices <= 0 || vertices > 20)
17     {
18         cout << "Invalid number of vertices!" << endl;
19         return 0;
20     }
21
22     // Create adjacency list (1-based)
23     vector<int> adj[21]; // Using index 1 to vertices
24
25     // Input number of edges
26     cout << "Enter number of edges: ";
27     cin >> edges;
28
29     // Maximum edges in undirected graph including self-loops = n*(n+1)/2
30     if (edges < 0 || edges > vertices * (vertices + 1) / 2)
31     {
32         cout << "Invalid number of edges!" << endl;
33         return 0;
34     }
35
36     // Input edges
37     for (int i = 0; i < edges; i++)
38     {
39         int u, v;
40         cout << "Enter edge " << i + 1 << " (vertex1 vertex2): ";
41         cin >> u >> v;
42
43         // Validate vertex numbers
44         if (u < 1 || v < 1 || u > vertices || v > vertices)
45         {
```

Figure 4: Graph Representation using Adjacency List Code -1

```

45     {
46         cout << "Invalid vertices! Enter values between 1 and " << vertices << endl;
47         i--; // retry this edge
48         continue;
49     }
50
51     // Undirected graph
52     adj[u].push_back(v);
53     if (u != v) // Avoid double entry for self-loop
54         adj[v].push_back(u);
55 }
56
57 // Display adjacency list
58 cout << "\nAdjacency List Representation:\n";
59 for (int i = 1; i <= vertices; i++)
60 {
61     cout << "V" << i << " -> ";
62     for (int j = 0; j < adj[i].size(); j++)
63     {
64         cout << "V" << adj[i][j] << " ";
65     }
66     cout << endl;
67 }
68
69 return 0;
70 }

```

Figure 5: Graph Representation using Adjacency List Code- 2

Output Placeholder:

```

C:\Users\hp\Desktop\2024-Sf  X  +  v

=====
                          Graph Representation using Adjancecy list
=====

Enter number of vertices (1 to 20): 5
Enter number of edges: 5
Enter edge 1 (vertex1 vertex2): 1 2
Enter edge 2 (vertex1 vertex2): 2 4
Enter edge 3 (vertex1 vertex2): 4 3
Enter edge 4 (vertex1 vertex2): 5 1
Enter edge 5 (vertex1 vertex2): 3 3

Adjacency List Representation:
V1 -> V2 V5
V2 -> V1 V4
V3 -> V4 V3
V4 -> V2 V3
V5 -> V1

-----
Process exited after 30.35 seconds with return value 0
Press any key to continue . . . |

```

Figure 6: Graph Representation using Adjacency List Output

Program 3: Breadth First Search (BFS) Traversal

This program performs BFS traversal starting from a user-defined vertex using a queue.

Code:

```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  int main()
7  {
8      int vertices, edges;
9
10     cout << "===== "<< endl;
11     cout << "          BFS traversal using queue "<< endl;
12     cout << "===== "<< endl;
13
14     // Input number of vertices
15     cout << "Enter number of vertices (1 to 20): ";
16     cin >> vertices;
17
18     // Validate vertices
19     if (vertices <= 0 || vertices > 20)
20     {
21         cout << "Invalid number of vertices!" << endl;
22         return 0;
23     }
24
25     // Adjacency list (1-based)
26     vector<int> adj[21];
27
28     // Input number of edges
29     cout << "Enter number of edges: ";
30     cin >> edges;
31
32     // Maximum edges including self-loops = n*(n+1)/2
33     if (edges < 0 || edges > vertices * (vertices + 1) / 2)
34     {
35         cout << "Invalid number of edges!" << endl;
36         return 0;
37     }
38
39     // Input edges
40     for (int i = 0; i < edges; i++)
41     {
42         int u, v;
43         cout << "Enter edge " << i + 1 << " (vertex1 vertex2): ";
44         cin >> u >> v;
45
46         // Validate vertices
47         if (u < 1 || v < 1 || u > vertices || v > vertices)
48         {
49             cout << "Invalid vertices! Enter values between 1 and " << vertices << endl;
50             i--; // retry this edge
51         }
52     }
53 }
```

Figure 7: Breadth First Search (BFS) Traversal Code -01


```

51         continue;
52     }
53     // Undirected graph
54     adj[u].push_back(v);
55     if (u != v) // Avoid double insertion for self-loop
56         adj[v].push_back(u);
57 }
58
59 int start;
60 cout << "Enter starting vertex for BFS (1 to " << vertices << "): ";
61 cin >> start;
62 // Validate starting vertex
63 if (start < 1 || start > vertices)
64 {
65     cout << "Invalid starting vertex!" << endl;
66     return 0;
67 }
68 // Visited array
69 bool visited[21] = {false};
70 // Queue for BFS
71 queue<int> q;
72 // Start BFS
73 visited[start] = true;
74 q.push(start);
75 cout << "\nBFS Traversal: ";
76 while (!q.empty())
77 {
78     int current = q.front();
79     q.pop();
80
81     cout << "V" << current << " ";
82     // Visit all adjacent vertices
83     for (int i = 0; i < adj[current].size(); i++)
84     {
85         int next = adj[current][i];
86         if (!visited[next])
87         {
88             visited[next] = true;
89             q.push(next);
90         }
91     }
92 }
93 cout << endl;
94 return 0;
95 }

```

Figure 9: Figure 8: Breadth First Search (BFS) Traversal using queue Code -02

Output:

```

C:\Users\hp\Desktop\2024-Sf  X  +  v
=====
BFS traversal using queue
=====
Enter number of vertices (1 to 20): 5
Enter number of edges: 6
Enter edge 1 (vertex1 vertex2): 3 1
Enter edge 2 (vertex1 vertex2): 2 3
Enter edge 3 (vertex1 vertex2): 4 5
Enter edge 4 (vertex1 vertex2): 3 4
Enter edge 5 (vertex1 vertex2): 2 2
Enter edge 6 (vertex1 vertex2): 1 5
Enter starting vertex for BFS (1 to 5): 2

BFS Traversal: V2 V3 V1 V4 V5

-----
Process exited after 36.9 seconds with return value 0
Press any key to continue . . . |

```

Figure 8: Figure 8: Breadth First Search (BFS) Traversal using queue Output

Program 4: Depth First Search (DFS) Traversal

This program performs DFS traversal using recursion, visiting a vertex completely before moving to the next.

Code:

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  // -----
6  // DFS Function using Recursion
7  // -----
8  // This function performs Depth-First Search starting from 'vertex'.
9  // It recursively visits all unvisited vertices connected to the current vertex.
10 void DFS(int vertex, vector<int> adj[], bool visited[])
11 {
12     // Mark current vertex as visited
13     visited[vertex] = true;
14
15     // Print the vertex (part of DFS traversal output)
16     cout << "V" << vertex << " ";
17
18     // Explore all adjacent vertices
19     for (int i = 0; i < adj[vertex].size(); i++)
20     {
21         int next = adj[vertex][i];
22
23         // If the adjacent vertex has not been visited, recursively call DFS
24         if (!visited[next])
25         {
26             DFS(next, adj, visited); // <-- Recursive call happens here
27         }
28     }
29 }
30 int main()
31 {
32     int vertices, edges;
33     cout << "===== "<<endl;
34     cout << "                DFS traversal using Recursion "<<endl;
35     cout << "===== "<<endl;
36     // Input the number of vertices in the graph
37     cout << "Enter number of vertices (1 to 20): ";
38     cin >> vertices;
39     // Validate vertices
40     if (vertices <= 0 || vertices > 20)
41     {
42         cout << "Invalid number of vertices!" << endl;
43         return 0;
44     }
45     // Create adjacency list for the graph (1-based indexing)
46     vector<int> adj[21]; // adj[1] to adj[vertices]
```

Figure 10: DFS Traversal using Recursion Code -01

```

47 // Input the number of edges
48 cout << "Enter number of edges: ";
49 cin >> edges;
50 // Input edges from the user
51 for (int i = 0; i < edges; i++)
52 {
53     int u, v;
54     cout << "Enter edge " << i + 1 << " (vertex1 vertex2): ";
55     cin >> u >> v;
56
57     // Validate vertex numbers
58     if (u < 1 || v < 1 || u > vertices || v > vertices)
59     {
60         cout << "Invalid vertices! Enter values between 1 and " << vertices << endl;
61         i--; // Retry this edge
62         continue;
63     }
64
65     // Add edge to adjacency List (undirected graph)
66     adj[u].push_back(v);
67     if (u != v) // Avoid duplicate entry for self-loop
68         adj[v].push_back(u);
69 }
70 // Input starting vertex for DFS traversal
71 int start;
72 cout << "Enter starting vertex for DFS (1 to " << vertices << "): ";
73 cin >> start;
74 // Validate starting vertex
75 if (start < 1 || start > vertices)
76 {
77     cout << "Invalid starting vertex!" << endl;
78     return 0;
79 }
80
81 // Visited array to keep track of visited vertices during DFS
82 bool visited[21] = {false};
83
84 cout << "\nDFS Traversal: ";
85
86 // Call DFS starting from 'start' vertex
87 DFS(start, adj, visited); // <-- This starts the recursive DFS
88
89 cout << endl;
90
91 return 0;

```

Figure 12: DFS Traversal using Recursion Code -02

Output:

```

C:\Users\hp\Desktop\2024-SE  X  +  v
=====
DFS traversal using Recursion
=====
Enter number of vertices (1 to 20): 5
Enter number of edges: 6
Enter edge 1 (vertex1 vertex2): 1 3
Enter edge 2 (vertex1 vertex2): 3 2
Enter edge 3 (vertex1 vertex2): 2 5
Enter edge 4 (vertex1 vertex2): 4 4
Enter edge 5 (vertex1 vertex2): 4 5
Enter edge 6 (vertex1 vertex2): 3 5
Enter starting vertex for DFS (1 to 5): 3

DFS Traversal: V3 V1 V2 V5 V4
=====

```

Figure 11: DFS Traversal using Recursion Output

Results & Observations

- Graphs can be represented efficiently using both adjacency matrix and adjacency list.
- BFS explores vertices in breadth-wise order using a queue.
- DFS explores vertices deeply using recursion.
- User-controlled input makes the programs flexible and reusable.

Conclusion

This Open-Ended Lab provided practical exposure to graph data structures and their traversal techniques. Through self-learning and implementation, a clear understanding of BFS and DFS algorithms was achieved. The lab improved problem-solving skills and reinforced concepts of non-linear data structures in C++.

Reflection

After completing this OEL, I am able to:

- Understand graph concepts and terminology
- Implement graph representations in C++
- Apply BFS and DFS traversal algorithms
- Develop clean and beginner-level graph programs